

Crafting Interpreters

Chapter 4 and Chapter 5 Challenges

Tyler Bacong

September 10, 2026

- 1 Submission Overview
- 2 Chapter 4 — Scanning
 - 2.1 Challenge 1 — Why Python and Haskell lexical grammars are not regular
 - 2.1.1 Python
 - 2.1.2 Haskell
 - 2.2 Challenge 2 — Places where spaces affect parsing
 - 2.2.1 CoffeeScript
 - 2.2.2 Ruby
 - 2.2.3 C preprocessor
 - 2.3 Challenge 3 — Why preserve comments and whitespace
 - 2.4 Challenge 4 — Nested C-style block comments in Lox
- 3 Chapter 5 — Representing Code
 - 3.1 Challenge 1 — Grammar without metasyntactic sugar
 - 3.1.1 Bonus — What the grammar represents
 - 3.2 Challenge 2 — A functional complement to Visitor
 - 3.3 Challenge 3 — Reverse Polish notation visitor
- 4 Verification and Code Map
- 5 References

\newpage

1 Submission Overview

This report completes the challenges at the ends of Chapter 4, "Scanning," and Chapter 5, "Representing Code," in Robert Nystrom's *Crafting Interpreters*. The code is separate from this report and is available at:

<https://github.com/TylerBac0ng/crafting-interpreters-ch4-5>

The implementation includes a Java scanner with nested C-style block comments, a Java reverse Polish notation visitor, automated Java tests, and a tested Racket implementation of the complementary functional pattern discussed in Chapter 5.

2 Chapter 4 — Scanning

2.1 Challenge 1 — Why Python and Haskell lexical grammars are not regular

A regular language can be recognized by a finite-state machine. Such a machine has only a fixed, finite amount of memory: it can remember which one of its finite states it is in, but it cannot maintain an arbitrarily deep stack or an unbounded counter. Regular expressions in the formal-language sense describe this same class of languages.

Python and Haskell use indentation to represent block structure. Their lexical analyzers must therefore remember earlier indentation levels rather than merely looking at the current character and a finite state.

2.1.1 Python

Python's tokenizer compares each logical line's indentation with a stack of previous indentation levels. A greater indentation pushes a level and emits an `INDENT` token. A smaller indentation must match an earlier level; the tokenizer pops levels and emits a `DEDENT` for each pop. Python's language reference explicitly specifies this stack algorithm ([Python lexical analysis, §2.1.8](#)).

Because a program can nest blocks to arbitrary depth in the language model, the stack can be arbitrarily deep. A finite automaton cannot store that stack, so the complete lexical process is not regular. An implementation may impose a finite nesting limit, but that practical limit does not make the intended language rule regular.

2.1.2 Haskell

Haskell's layout or "off-side" rule has the same fundamental issue. After `where`, `let`, `do`, or `of`, the lexer/layout stage remembers an indentation column. Later indentation can cause implicit `{`, `;`, and `}` tokens to be inserted. Nested layout regions require a stack of layout contexts. The formal Haskell 2010 rule explicitly takes a stack as input ([Haskell 2010 Report, §10.3](#)).

Therefore, both languages need unbounded context about prior indentation. That extra memory places their full lexical/layout processing beyond regular languages.

2.2 Challenge 2 — Places where spaces affect parsing

2.2.1 CoffeeScript

CoffeeScript makes both call parentheses and an empty parameter list on an anonymous function optional. This creates an ambiguity that whitespace resolves:

```
someFunction() -> someLambda  
someFunction () -> someLambda
```

The first form calls `someFunction` with no arguments, then calls the returned function with a lambda argument. In JavaScript-like parentheses, its structure is:

```
someFunction()(( ) -> someLambda)
```

The space in the second form makes the lambda the argument of `someFunction`:

```
someFunction(( ) -> someLambda)
```

This corner follows from CoffeeScript's two conveniences: indentation-sensitive syntax and calls without parentheses. Both are documented in the official [CoffeeScript language reference](#).

2.2.2 Ruby

Ruby also permits method calls without parentheses, so spacing can determine whether `-` is binary subtraction or unary negation attached to a method argument. For example:

```
elements.count - 1  # subtract 1 from the count  
elements.count -1   # call count with the argument -1
```

If `elements` is `[-1, 4, 9]`, the first expression evaluates to 2; the second evaluates to 1, the number of elements equal to `-1`. Ruby's parser even contains diagnostics for an "ambiguous first argument" involving unary `+` and `-` ([Ruby parser diagnostics](#)). The ambiguity exists because parentheses around method arguments are optional, as described by the official [Ruby method-call syntax](#).

2.2.3 C preprocessor

In a macro definition, there must be no space between a function-like macro's name and its opening parenthesis:

```
#define SQUARE(x) ((x) * (x)) /* function-like macro */  
#define SQUARE (x) ((x) * (x)) /* object-like macro */
```

The first definition declares one parameter named `x`. The second defines an object-like macro named `SQUARE` whose replacement text happens to begin with `(x)`. This is not cosmetic formatting; the two definitions have different token-level semantics. The GNU C preprocessor manual states this distinction directly ([GCC, "Function-like Macros"](#)).

2.3 Challenge 3 — Why preserve comments and whitespace

A compiler that only needs to execute a program can usually discard comments and insignificant whitespace. Many other tools need the original “trivia,” however:

1. **Formatters and pretty-printers** must preserve comments and may retain deliberate blank lines, wrapping, or alignment.
2. **Documentation generators** need documentation comments attached to the declaration immediately following them.
3. **IDEs and language servers** display hover documentation, comments, exact source ranges, and refactoring previews.
4. **Source-to-source tools** such as automated refactoring, migration, and transpilation should not erase a programmer’s explanations.
5. **Linters** may enforce comment conventions, copyright headers, disabled-code markers, or spacing rules.
6. **Concrete syntax trees and incremental parsers** need every character so a small edit can be applied without reconstructing the whole file.
7. **Round-trip serialization** requires the property that printing an unchanged syntax tree reproduces the original source exactly.

One design is to emit whitespace and comments as ordinary tokens. Another is to attach leading and trailing trivia to nearby meaningful tokens. The second approach keeps the parser’s main token stream clean while preserving the full source for tools that need it.

2.4 Challenge 4 — Nested C-style block comments in Lox

I extended the Chapter 4 Java scanner so `/` has three cases:

- `//` consumes a line comment;
- `/*` enters the new block-comment scanner; and
- a lone `/` still emits the division token.

The main addition uses a nesting-depth counter:

```

private void blockComment() {
    int depth = 1;

    while (depth > 0 && !isAtEnd()) {
        if (peek() == '/' && peekNext() == '*') {
            advance();
            advance();
            depth++;
        } else if (peek() == '*' && peekNext() == '/') {
            advance();
            advance();
            depth--;
        } else {
            if (peek() == '\n') line++;
            advance();
        }
    }

    if (depth > 0) {
        errors.add("[line " + line + "] Error: Unterminated block comment.");
    }
}

```

Every nested `/*` increments `depth`, and every `*/` decrements it. The comment ends only when the counter returns to zero. Newlines inside comments increment the scanner's line number, so the next real token and any later diagnostics still report the correct source line. Reaching end-of-file with a positive `depth` produces an unterminated-comment error.

Nesting required only a small amount of Java code, so it was not mechanically difficult. Conceptually, however, it changes the recognizer: stopping at the first `*/` needs no nesting memory, while arbitrary nesting requires an unbounded counter. Like indentation in Challenge 1, the nested-comment language is not regular unless the implementation imposes a fixed maximum depth.

The automated tests cover nested comments across multiple lines, correct token line numbers, unterminated comments, EOF recovery, and preservation of ordinary division. The verified result was:

```
All Chapter 4-5 Java tests passed.
```

3 Chapter 5 — Representing Code

3.1 Challenge 1 — Grammar without metasyntactic sugar

The original grammar is:

```
expr → expr ( "(" ( expr ( "," expr )* )? ")" | "." IDENTIFIER )+  
      | IDENTIFIER  
      | NUMBER
```

The following grammar matches the same language without using `|`, `*`, `+`, or `?`. Repeating a production name expresses alternatives, recursion expresses repetition, and ϵ means the empty string.

```
expr    → expr calls  
expr    → IDENTIFIER  
expr    → NUMBER  
  
calls   → calls call  
calls   → call  
  
call    → "(" "  
call    → "(" arguments "  
call    → "." IDENTIFIER  
  
arguments → expr  
arguments → arguments "," expr
```

Why it is equivalent:

- The three `expr` rules replace the original top-level `|` alternatives.
- `calls → calls call` plus `calls → call` means one or more calls, replacing the original `+`.
- Separate empty-argument and nonempty-argument call rules replace `?`.
- The recursive `arguments` rule replaces the comma-separated `*` repetition.

3.1.1 Bonus — What the grammar represents

It represents identifiers and numbers followed by one or more function-call or property-access suffixes. In other words, it describes chained calls and member access, such as:

```
breakfast(eggs, bacon).withToast().price
```

The grammar is left-recursive because every new suffix applies to the complete expression built before it.

3.2 Challenge 2 — A functional complement to Visitor

With algebraic data types and pattern matching, a functional program typically groups code by operation: one evaluate function contains a case for every node type, another print function contains another case for every node type, and so on. This is easy to extend with a new operation but requires changing every operation when a new node type is introduced. Visitor gives an object-oriented language that same organization.

The complementary pattern should reverse that tradeoff in a functional language. I implemented each expression node as a closure that receives an operation message. The closure captures its node data and bundles every operation for that node type:

```
(define (literal value)
  (lambda (operation)
    (case operation
      [(evaluate) value]
      [(pretty) (format "~a" value)]
      [(rpn) (format "~a" value)])))

(define (binary left operator right)
  (lambda (operation)
    (case operation
      [(evaluate) ...]
      [(pretty) ...]
      [(rpn) ...])))
```

Clients send messages without knowing the hidden representation:

```
(define (evaluate expression) (expression 'evaluate))
(define (pretty-print expression) (expression 'pretty))
(define (to-rpn expression) (expression 'rpn))
```

This is an object encoded with functions: captured variables act like private fields, and the message-dispatch closure acts like a method table. A new node type is easy to add because it supplies a new constructor/closure implementing the existing messages; existing client functions remain unchanged. The mirror tradeoff is that adding a new operation requires updating every node constructor. That is exactly complementary to the usual functional layout.

The complete implementation is in `src/racket/functional_nodes.rkt`. I ran it with Racket 9.3, including its rackunit tests:

```
3 tests passed
((1 + 2) * (4 - 3))
1 2 + 4 3 - *
3
```

3.3 Challenge 3 — Reverse Polish notation visitor

The RPN visitor performs a postorder traversal. A binary expression visits the left operand, then the right operand, then emits the operator. Grouping emits no symbol because RPN order is already unambiguous.

```
final class RpnPrinter implements Expr.Visitor<String> {
    String print(Expr expression) {
        return expression.accept(this);
    }

    @Override
    public String visitBinaryExpr(Expr.Binary expression) {
        return expression.left.accept(this) + " " +
            expression.right.accept(this) + " " + expression.operator.lexeme;
    }

    @Override
    public String visitGroupingExpr(Expr.Grouping expression) {
        return expression.expression.accept(this);
    }

    @Override
    public String visitLiteralExpr(Expr.Literal expression) {
        return expression.value == null ? "nil" : expression.value.toString();
    }

    @Override
    public String visitUnaryExpr(Expr.Unary expression) {
        String operator = expression.operator.type == TokenType.MINUS
            ? "~" : expression.operator.lexeme;
        return expression.right.accept(this) + " " + operator;
    }
}
```

For the requested expression, the constructed syntax tree is equivalent to:



A postorder walk visits the nodes in this order:

1, 2, +, 4, 3, -, *

Therefore:

$(1 + 2) * (4 - 3) \rightarrow 1\ 2\ +\ 4\ 3\ -\ *$

I use ~ for unary negation. If both unary and binary minus emitted -, an RPN evaluator would not know whether to pop one operand or two. The automated test therefore also verifies:

`-123 * ("str") → 123 ~ str *`

4 Verification and Code Map

The checked-in Makefile builds the Java sources and runs the Java assertions. The Racket target runs the functional-pattern tests.

```
> mingw32-make clean test
All Chapter 4-5 Java tests passed.

> raco test src/racket/functional_nodes.rkt
3 tests passed
```

Requirement	Code file
Nested block-comment scanner	chapters-4-5/src/java/com/craftinginterpreters/lox/Scanner.java
Scanner and RPN tests	chapters-4-5/src/java/com/craftinginterpreters/lox/RunTests.java
Expression syntax-tree classes	chapters-4-5/src/java/com/craftinginterpreters/lox/Expr.java
RPN visitor	chapters-4-5/src/java/com/craftinginterpreters/lox/RpnPrinter.java
Functional complementary pattern	chapters-4-5/src/racket/functional_nodes.rkt
Reproducible build	chapters-4-5/Makefile

5 References

Nystrom, R. "Scanning." *Crafting Interpreters*. <https://craftinginterpreters.com/scanning.html>

Nystrom, R. "Representing Code." *Crafting Interpreters*. <https://craftinginterpreters.com/representing-code.html>

Python Software Foundation. "Lexical Analysis — Indentation." https://docs.python.org/3/reference/lexical_analysis.html#indentation

Haskell.org. "Haskell 2010 Language Report — Layout." <https://www.haskell.org/onlinereport/haskell2010/haskellch10.html#x17-17800010.3>

CoffeeScript.org. "CoffeeScript Language Reference." <https://coffeescript.org/#language>

Ruby Documentation. "Calling Methods." https://ruby-doc.org/3.2.2/syntax/calling_methods_rdoc.html

GNU Project. "The C Preprocessor — Function-like Macros." <https://gcc.gnu.org/onlinedocs/cpp/Function-like-Macros.html>